SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4304

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I Venkata Chaithanya.R (192471027)____) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Venkata Chaithanya R

1. Online Symposium Portal

The symposium registration page is functional at a basic level, but its structure can be strengthened by introducing semantic HTML, accessible form controls, browser validation, and a clearly defined request-response flow.

1.1 Identify suitable HTML5 semantic elements missing from the form

The original form mainly contains input controls and a submit button. For a complete webpage, the form should be placed inside meaningful structural elements. The following elements are useful:

- <header> – introduces the symposium and can contain its title, event date, or introductory message.
- <main> – identifies the central content that the student is expected to interact with.
- <section> – separates the registration area from other possible page content.
- <footer> – can hold contact details, copyright information, or college support information.
- <label> – gives every input a clear accessible name and improves usability for screen-reader users.
- <fieldset> and <legend> – group registration fields under a meaningful heading.

Using these elements creates a logical page hierarchy rather than treating the entire interface as an unstructured collection of controls.

1.2 Modify the structure to make it standards-compliant

The following version adds a complete HTML5 document structure and also introduces an event category field. The extra field demonstrates how the registration form can be extended without making the structure complicated.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Symposium Registration</title>
</head>
<body>

<header>
  <h1>College Symposium 2026</h1>
  <p>Register online to participate in the technical event.</p>
</header>

<main>
  <section aria-labelledby="register-title">
    <h2 id="register-title">Student Registration</h2>

    <form action="/register" method="post">
      <fieldset>
        <legend>Participant Information</legend>

        <label for="name">Full Name:</label>
        <input type="text" id="name" name="name"
               autocomplete="name" required>

        <label for="email">Email Address:</label>
        <input type="email" id="email" name="email"
               autocomplete="email" required>

        <label for="event">Event Category:</label>
        <select id="event" name="event" required>
          <option value="">Select an event</option>
          <option value="paper">Paper Presentation</option>
          <option value="quiz">Technical Quiz</option>
          <option value="project">Project Expo</option>
        </select>
```

```
            <button type="submit">Complete Registration</button>
        </fieldset>
      </form>
    </section>
</main>

<footer>
    <p>College Symposium Support | 2026</p>
</footer>

</body>
</html>
```

The revised version includes a document type declaration, language information, responsive metadata, explicit labels, required fields, and a meaningful event-selection control. These changes improve both technical quality and the user experience.

1.3 Interpret the HTTP request produced by the submission

After the participant completes the fields and activates the submit button, the browser packages the successful controls and sends them to the address specified in the action attribute. Since the form uses POST, the submitted values are carried in the request body.

```
POST /register HTTP/1.1
Host: symposium.example
Content-Type: application/x-www-form-urlencoded

name=Ranjith&email=student%40example.com&event=project
```

Here, POST identifies the operation, /register identifies the server-side endpoint, Host identifies the destination server, and Content-Type describes the encoding of the submitted data. The server can then validate the information and create a registration record.

1.4 Analyze the HTTP response cycle

The communication can be understood as a sequence of events:

1. The student enters the required registration information.
2. The browser performs basic form validation and creates a POST request.

3. The web server receives the request at the /register endpoint.
4. The application checks the submitted values and performs business processing, such as saving the registration.
5. The server creates a response containing a status code, headers, and possibly an HTML body.
6. The browser interprets the response and presents the appropriate result to the student.

A successful application may return a confirmation page directly with HTTP 200 OK, or it may redirect the user to a separate confirmation page. A redirect-based design can prevent accidental duplicate submissions when a user refreshes the final page.

```
HTTP/1.1 303 See Other
Location: /registration-success
```

1.5 Improvements for usability and accessibility

- Use visible labels instead of depending on placeholder text alone.
- Mark mandatory fields with required and explain the requirement clearly.
- Keep validation messages specific, for example, 'Enter a valid college email address.'
- Place the keyboard focus on the first invalid field after validation.
- Maintain a logical tab order and make all controls keyboard accessible.
- Use descriptive button text such as 'Complete Registration' rather than a vague label.
- Provide sufficient text contrast and do not use colour as the only indication of an error.
- Make the layout responsive for mobile phones, tablets, and desktop screens.
- Preserve previously entered values when a submission fails.
- Provide a clear confirmation message or registration reference after successful submission.

2. Cross-Browser Compatibility Issue

The supplied page contains incomplete markup. Modern browsers can often recover from HTML errors, but relying on automatic error correction can produce an unexpected document tree and inconsistent behaviour when CSS or JavaScript is added.

2.1 Identify improper nesting and missing closing tags

The heading is opened but not explicitly closed, and the paragraph is also left open. The problematic structure is:

```
<div>
  <h1>Welcome
  <p>About us
</div>
```

The heading should be closed before the paragraph begins, and the paragraph should also have its closing tag. A minimally corrected version is:

```
<div>
  <h1>Welcome</h1>
  <p>About us</p>
</div>
```

Explicit closing tags make the intended parent-child relationship clear to browsers, validators, assistive technologies, and developers.

2.2 How browsers may interpret the malformed markup

HTML parsers include error-recovery rules. When markup is incomplete, the browser attempts to construct a usable DOM tree by implicitly closing or inserting elements. This is helpful for displaying many real-world pages, but it does not guarantee that the generated structure will match the developer's intention.

- CSS selectors may match elements differently from what the developer expected.
- JavaScript that traverses or modifies the DOM may operate on an unexpected structure.
- Spacing, margins, and layout can appear inconsistent when styles depend on element relationships.
- Accessibility tools may receive a less meaningful document hierarchy.

- Future browser changes or different rendering modes can expose errors that were previously unnoticed.

For these reasons, valid markup should be treated as the starting point for reliable cross-browser development.

2.3 Identify missing semantic elements

The generic div does not communicate the role of the content. A more meaningful organization would be:

- <header> for the company title or introductory content.
- <main> for the primary information displayed to visitors.
- <section> for a thematic block such as the company's profile.
- <footer> for contact, copyright, or other supporting information.

Semantic structure is useful not only for browsers but also for search engines, assistive technologies, and developers maintaining the website.

2.4 Corrected standards-based HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Company Profile</title>
</head>
<body>

<header>
    <h1>Welcome to Our Company</h1>
</header>

<main>
    <section aria-labelledby="about-title">
        <h2 id="about-title">About Our Company</h2>
        <p>We provide reliable digital services and customer-focused solutions.</p>
    </section>
</main>

<footer>
    <p>Contact the company support team for more information.</p>
</footer>

</body>
</html>
```

This structure establishes a clear hierarchy, closes every element correctly, and supplies the browser with the information needed to interpret the document in standards mode.

2.5 Techniques for improving cross-browser compatibility

- Start every HTML document with the HTML5 DOCTYPE.
- Use standards-compliant HTML and CSS instead of browser-specific markup.
- Validate the page during development to detect structural errors early.
- Test the page in more than one browser and at different viewport widths.
- Use responsive CSS and media queries for different screen sizes.
- Prefer feature detection over assumptions about a particular browser.
- Use progressive enhancement so essential content works before advanced features are added.
- Avoid obsolete HTML elements and unsupported vendor-specific techniques unless a compatibility plan exists.
- Check keyboard navigation and accessibility behaviour during browser testing.
- Use browser developer tools to compare the DOM, computed styles, and layout when a rendering difference appears.

3. Form Submission Failure

The login form does not behave as intended because the button has been defined as a general-purpose button. In addition, the choice of GET creates a security problem when credentials are involved.

3.1 Identify issues in the form configuration

The most direct problem is:

```
<button type="button">Login</button>
```

The type='button' value does not invoke the form's normal submission process. The form also uses GET, which is unsuitable for placing passwords in a request URL. Further improvements are needed for accessibility and validation.

- The username and password controls should have associated labels.
- required should be used when both values are mandatory.
- autocomplete should identify the purpose of the username and current password fields.
- The login endpoint must be served over HTTPS to protect credentials during transmission.

3.2 Why is the HTTP request not triggered?

Clicking a button whose type is button only activates that button; it does not submit its parent form automatically. Therefore, the browser does not begin the normal form-submission process.

Changing the control to a submit button fixes the immediate problem:

```
<button type="submit">Login</button>
```

A JavaScript event handler could also submit the form programmatically, but this is unnecessary when ordinary form submission is sufficient. A native submit button is simpler and works naturally with keyboard interaction and browser validation.

3.3 Interpret the role of GET in this scenario

GET is primarily used when a client requests information from a server. When a form uses GET, the form controls are encoded into the query portion of the URL.

```
GET /submit?username=student&password=example HTTP/1.1
```

This behaviour is useful for non-sensitive operations such as search or filtering, where sharing or bookmarking the resulting URL may be desirable. It is not appropriate for login credentials because the URL can appear in browser history, server or proxy logs, bookmarks, analytics records, or copied links.

For authentication, POST should be used together with HTTPS. POST by itself does not encrypt data; HTTPS provides the encryption during transport.

3.4 Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Student Login</title>
</head>
<body>

<main>
   <section aria-labelledby="login-title">
      <h1 id="login-title">Student Login</h1>

      <form action="/submit" method="post">
         <label for="username">Username:</label>
         <input type="text"
              id="username"
              name="username"
              autocomplete="username"
              required>

         <label for="password">Password:</label>
         <input type="password"
              id="password"
              name="password"
              autocomplete="current-password"
              required>

         <button type="submit">Sign In</button>
      </form>
```

```
  </section>
</main>

</body>
</html>
```

The corrected version uses POST, a native submit button, accessible labels, required validation, and suitable autocomplete values. These changes address both the functional problem and several usability concerns.

3.5 Improvements for secure data transmission

- Use HTTPS for the complete login journey, including the page that displays the form.
- Use POST so credentials are not placed in the URL query string.
- Perform server-side validation even when client-side validation is present.
- Never store passwords as plain text; use a suitable password-hashing mechanism on the server.
- Use secure session handling after authentication and protect authentication cookies with appropriate attributes.
- Apply rate limiting or account-protection controls against repeated automated login attempts.
- Do not place passwords in URLs, client-side debug output, or unnecessary application logs.
- Use appropriate security headers and keep the web framework and dependencies updated.
- Return generic authentication failure messages where appropriate so attackers cannot easily discover whether a username exists.

Overall Summary

The three cases demonstrate how correct HTML structure, HTTP behaviour, accessibility, and security are connected. Semantic HTML makes a page easier to understand and maintain. Correctly formed markup reduces browser inconsistencies. Understanding the difference between button types explains why a form may fail to submit. Finally, choosing POST and HTTPS for authentication helps protect sensitive information while it travels between the browser and server.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand